

Performance Analysis of Genetic Algorithms compared to Hill Climbing and Simulated Annealing on Benchmark numerical Functions

Olivia-Alexandra Gavril

November 2025

Abstract

This paper evaluates the performance of *Genetic Algorithms (GA)* in finding the minimum values for four standard test functions: De Jong, Schwefel, Rastrigin and Michalewicz. The study investigates three different problem sizes, or dimensions ($D = 5, 10, \text{ and } 30$). Our primary goal is to compare the GA's results against two simpler local search methods: *Hill Climbing* and *Simulated Annealing*. The experiments show that while the GA achieves effective optimization in the lower-dimensional problems ($D = 5, 10$), its performance degrades notably when faced with the difficult, high-dimensional landscapes ($D = 30$). The comparative analysis confirms that in the most complex scenarios, the GA is ultimately less effective than the tested variants of Hill Climbing and Simulated Annealing.

1 Introduction

This paper initiates a detailed assessment of the optimization performance achieved by Genetic Algorithms (GAs) across four benchmark numerical functions: De Jong, Schwefel's, Rastrigin's, and Michalewicz's functions. These mathematical models were selected to impose varying levels of difficulty, from simple convexity to intense multimodality, thereby providing an exacting environment to measure the GA's consistency and effectiveness. The central objective is to validate how successfully evolutionary computation tackles these diverse optimization terrains. In addition, this study follows a direct comparative approach, evaluating the Genetic Algorithm (GA) alongside well-known local search strategies, including the Hill Climbing (HC) variants and the Simulated Annealing (SA) metaheuristic, all tested on the same set of benchmark functions. The Genetic Algorithm mimics natural selection, allowing fitter solutions to influence future generations and gradually steer the population toward better optima, while Hill Climbing and Simulated Annealing rely on single-solution trajectories, HC using purely greedy moves and SA introducing controlled randomness, to explore the search space from a local-search perspective. To ensure a fair and consistent comparison, every algorithm is run under identical experimental settings, allowing us to assess differences in performance under matched conditions and to draw clear, reliable conclusions about their relative effectiveness.

2 Methods and Implementation

Solution representation

In the context of Genetic Algorithms, each candidate solution is represented as a bitstring. Since the benchmark functions (De Jong, Schwefel, Rastrigin, Michalewicz) take as input an array of real numbers, it is necessary to encode these values into a discrete form, suitable for binary representation.

The search space is divided according to a chosen precision, denoted as 10^{-d} . Each variable's interval $[a, b]$ is split into equal segments of this size, which determines the number of bits needed to represent all possible values, computed as:

$$N = (b - a) \times 10^d$$

Before evaluating a solution with the optimisation function, every binary-encoded variable must be converted back into its real form using the formula:

$$X_{\text{real}} = a + \frac{\text{decimal}(x_{\text{bits}}) \times (b - a)}{2^n - 1}$$

This encoding allows the algorithm to uniformly sample and evaluate the search space. Each bitstring uniquely defines a candidate solution, while the decoding equation rescales the binary value to its real counterpart within $[a, b]$, making it suitable for function evaluation.

Genetic Algorithm

Genetic Algorithms are optimization techniques inspired by the principles of natural selection and biological evolution. They operate iteratively on a population of candidate solutions, which are typically encoded as fixed-length binary strings (chromosomes) representing points in the solution space. The quality of each individual solution is quantified by a fitness function, and converts it into a single numerical score where higher scores are always better. The algorithm progresses through successive generations via three fundamental genetic operators, and a critical preservation mechanism: Selection, Crossover, Mutation and Elitism. [4]

For the **Selection** mechanism, the strategy was adapted based on problem dimensionality. **Roulette Wheel Selection** was used for problems with lower dimensions ($D = 5$ and $D = 10$) and it is switched to **Tournament Selection** for the more complex high-dimensional setting ($D = 30$).

- **Roulette Wheel Selection:** This method operates by treating the population as a segmented wheel, where the size of each individual's segment is directly proportional to its fitness. This ensures that fitter solutions have a higher probability of being chosen for reproduction.
- **Tournament Selection:** Conversely, this method involves randomly selecting a small group of individuals (the *tournament*) from the population; the individual with the best fitness from that group is then chosen as the parent. This approach applies less selection pressure than Roulette Wheel Selection, which is beneficial for maintaining greater diversity in the expansive $D = 30$ search space.

Crossover, the primary search operator, functions by combining genetic material from two parent chromosomes to produce offspring. This critical process introduces variation by merging characteristics from both parents, which effectively helps spread beneficial solution components throughout the population, thus driving exploitation. The secondary operator is **Mutation**, which randomly alters two bits in a chromosome, operating with a specific low probability. Mutation's central role is to maintain diversity within the population, acting as the main mechanism for exploration, preventing premature convergence and enabling the discovery of new regions of the search space.[7]

Elitism helps the algorithm keep the best solutions by copying a small number of top individuals directly into the next generation, without applying crossover or mutation to them. This prevents the best solution found so far from being lost due to randomness and ensures that the population's best fitness does not decrease. [5]

As the generations progress, selection, variation, and elitism gradually guide the population toward better solutions. The process continues until the algorithm meets a stopping condition or reaches the maximum number of generations.

3 Experiments and Results

Experimental Setup

All benchmark functions were optimized 30 times under three problem dimensionalities ($D = 5, 10, 30$), using a binary encoding scheme with a fixed precision of five decimal places. The algorithm changes how it works based on the size of the problem. For problems with fewer dimensions, we use the roulette-wheel selection method and keep more of the best individuals (a higher elitism rate). For the harder problems with more dimensions, we switch to tournament selection and save fewer best individuals (a reduced elitism level). The following parameters and values are used throughout the experiments:

- Population size: 200 (DeJong, Schwefel, Rastrigin, Michalewicz), 350 (Rastrigin, dimension=30);
- Crossover rate: 0.85 (DeJong, Schwefel, Michalewicz), 0.80(Rastrigin), 0.75(Rastrigin dimension=30);
- Mutation rate: 0.012 (DeJong), 0.025 (Schwefel), 0.035 (Rastrigin), 0.030(Rastrigin dimension=30), 0.045 (Michalewicz);
- Chromosomes selected in elitism:
 - $D \leq 10$: The elitism rate is set to **15%** (forDe Jong) and **12%** (for other functions).
 - $D = 30$: The elitism rate is reduced to **10%** (for De Jong) and **5%** (Rastrigin dimension=30) and **8%** (for other functions), minimizing the risk of premature convergence.
- Maximum generations: 2500 (DeJong), 3000 (Schwefel, Rastrigin, Michalewicz), 7000 (Rastrigin dimension=30);

The parameter values were chosen in this manner to ensure robust performance across varying problem complexities. A fixed *Population Size* maintains consistent diversity. The generally high *Crossover Rate* (0.80–0.85) emphasizes exploitation through recombination. Conversely, the *Mutation Rate* is progressively increased for complex, multimodal functions to guarantee sufficient exploration and help the algorithm escape local optima. Finally, the *Maximum Generations* are higher for complex functions, providing adequate time for full convergence. Due to the extremely multimodal nature of the Rastrigin function, which led to poor initial performance, specific adjustments were applied at $D=30$, with the cost of increased computational time.

3.1 De Jong’s Function

So called *first function of De Jong’s* is one of the simplest test benchmark. [3] The function is continuous, convex and unimodal. General function definition and restrictions:

$$f_1(\mathbf{x}) = \sum_{i=1}^n x_i^2, \quad -5.12 \leq x_i \leq 5.12 \quad (1)$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \dots, n$.

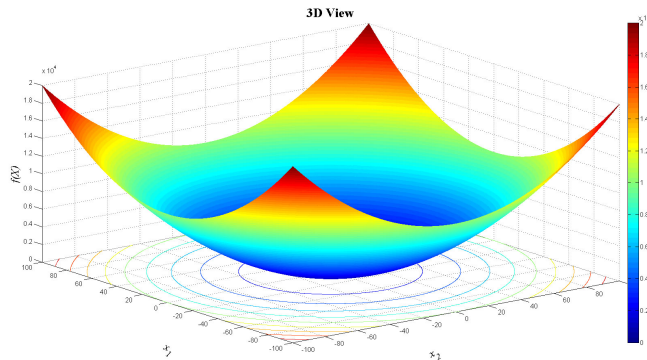


Figure 1: Visualisation of De Jong’s function, with a surface plot for variables in the range $[-500, 500]$.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00000	0.00000	0.00000	0.00000
Dimension = 30	0.00001	0.00006	0.00003	0.00001

Table 1: Statistics for different dimensions using the Genetic Algorithm on De Jongs function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	14.64682	20.42869	16.15386	1.27019
Dimension = 10	24.55145	28.15953	27.22605	0.73739
Dimension = 30	276.93575	283.87172	278.5448	1.16416

Table 2: Statistics for the running time of the Genetic Algorithm(GA) on De Jongs function, in seconds.

Algorithm	Dimension = 5				Dimension = 10			
	Min	Max	Mean	Stdev	Min	Max	Mean	Stdev
HC: FI	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
HC: BI	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.000000	0.00000
HC: WI	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
SA	0.00000	0.04287	0.00849	0.01446	0.00000	0.04723	0.00319	0.00970

Table 3: Heuristic performance of Hill Climbing (HC) and Simulated Annealing (SA) for dimensions 5 and 10 on De Jongs function.

Algorithm	Dimension = 30			
	Min	Max	Mean	Stdev
HC: FI	0.00000	0.00000	0.00000	0.00000
HC: BI	0.00000	0.00000	0.00000	0.00000
HC: WI	0.00000	0.00000	0.00000	0.00000
SA	0.00000	0.00001	0.00000	0.00000

Table 4: Heuristic of Hill Climbing (HC) and Simulated Annealing (SA) performance for dimension 30 on De Jongs function.

For smaller dimensions, the Genetic Algorithm (GA) and Hill Climbing (HC) variants achieve perfect results with zero deviation, indicating high stability. Simulated Annealing (SA) shows slightly higher variability but still reaches near-optimal solutions. For bigger dimensions, only HC and SA remain stable and perform better than GA.

3.2 Schwefel's Function

Schwefel's function is particularly deceptive because its global minimum is located far away from most local minima, making it challenging for optimization algorithms to locate the true optimum. The function has the following definition and restrictions:

$$f_7(\mathbf{x}) = \sum_{i=1}^n -x_i \cdot \sin\left(\sqrt{|x_i|}\right), \quad -500 \leq x_i \leq 500 \quad (2)$$

Its global minimum $f(x) = -418.9829n$ is obtainable for $x_i = 420.9687$, where $i = 1, \dots, n$.

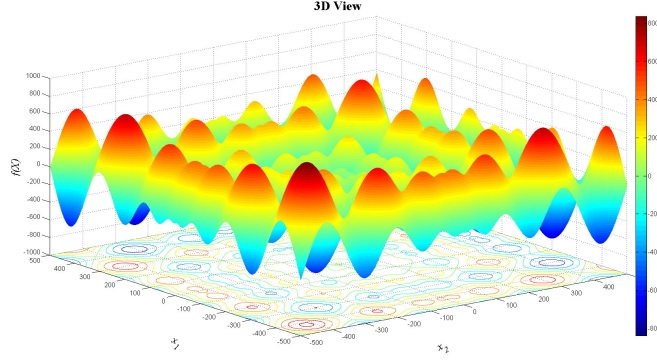


Figure 2: Visualization of Schwefel's function with a surface plot for variables in the range $[-500, 500]$.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-2094.91444	-2094.70648	-2094.89	0.05221
Dimension = 10	-4188.99778	-4189.82630	-4189.415	0.20996
Dimension = 30	-12392.81396	-11537.29631	-12056.95	176.7721

Table 5: Statistics for different dimensions using the Genetic Algorithm on Schwefel's function.

GA running time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	28.34499	35.36355	28.94331	1.34845
Dimension = 10	44.54907	45.35387	44.89211	0.18126
Dimension = 30	403.50527	407.87313	405.5833	0.96331

Table 6: Statistics for the running time of the Genetic Algorithm(GA) on Schwefel's function, in seconds.

Algorithm	Dimension = 5				Dimension = 10			
	Min	Max	Mean	Stdev	Min	Max	Mean	Stdev
HC: FI	-1895.321	-1142.963	-1541.724	186.865	-3561.499	-2736.975	-3135.864	208.3373
HC: BI	-2094.707	-1263.014	-1691.276	197.6043	-3941.593	-2825.47	-3307.447	267.5968
HC: WI	-2006.809	-1230.29	-1544.643	174.5579	-3617.15	-2676.319	-3179.849	249.318
SA	-2094.913	-2094.496	-2094.712	0.09906	-4189.619	-4188.999	-4189.359	0.15066

Table 7: Heuristic performance of Hill Climbing (HC) and Simulated Annealing (SA) for dimensions 5 and 10 on Schwefel's function

Algorithm	Dimension = 30			
	Min	Max	Mean	Stdev
HC: FI	-10656.63	-8276.408	-9355.231	549.099
HC: BI	-11381.79	-9062.042	-10091.11	527.2629
HC: WI	-10244.87	-8641.939	-9292.544	436.5778
SA	-12568.44	-12166	-12413.02	126.4578

Table 8: Heuristic of Hill Climbing (HC) and Simulated Annealing (SA) performance for dimension 30 on Schwefel's function

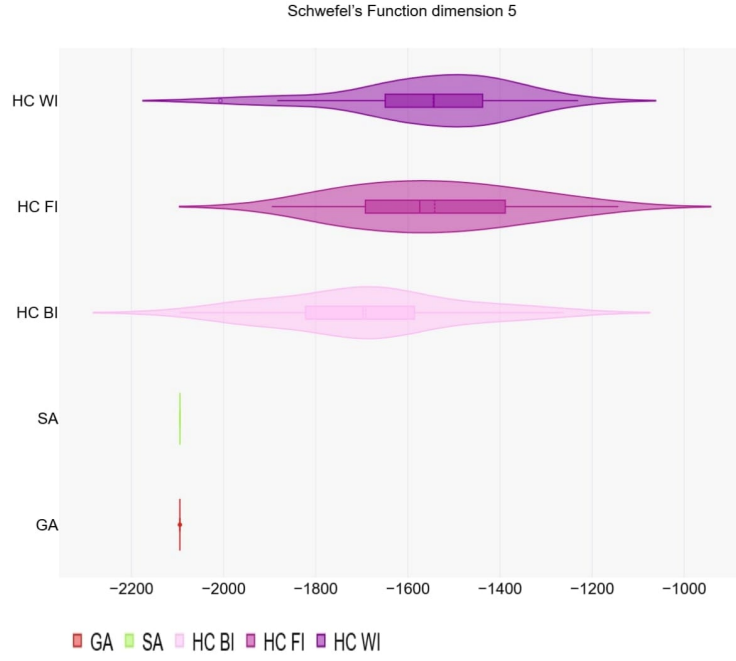


Figure 3: Violin Plot For the algorithms tested on Schwefel's function, dimension 5.

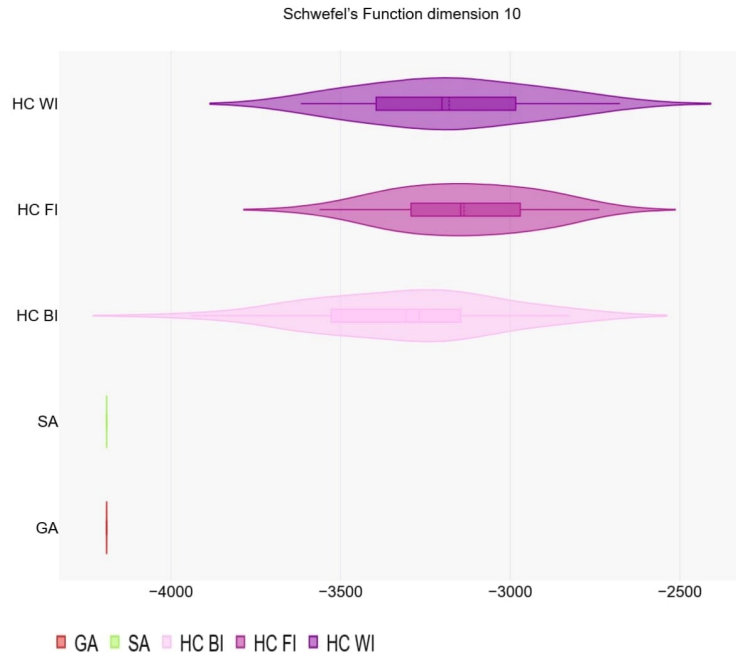


Figure 4: Violin Plot For the algorithms tested on Schwefel's function, dimension 10.

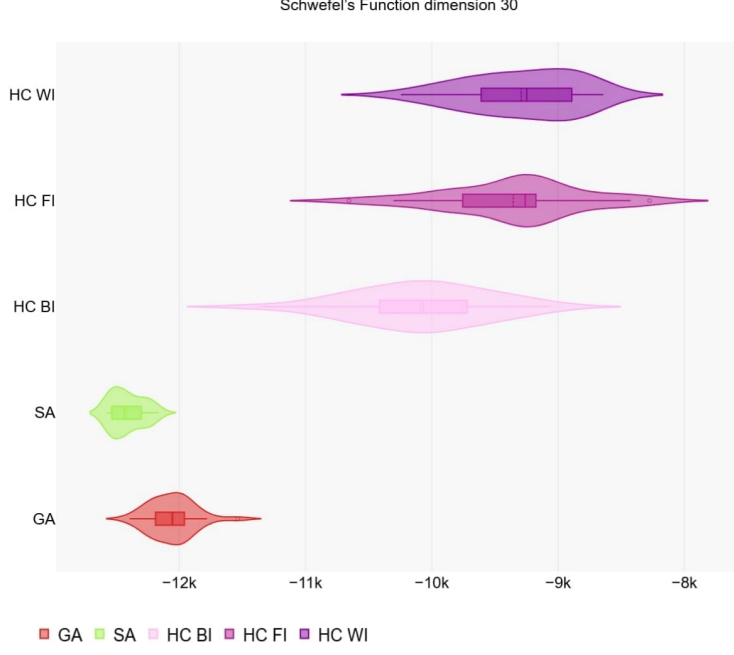


Figure 5: Violin Plot For the algorithms tested on Schwefel's function, dimension 30.

The Genetic Algorithm (GA) and Simulated Annealing (SA) both approach the global optimum at dimension 5, with GA showing slightly higher stability. At dimension 30, SA outperforms GA, achieving lower values, demonstrating better ability to escape local minima. Hill Climbing (HC), in all variants, struggles to reach the optimum, exhibiting much higher mean values and variability, confirming its limitations on complex, multimodal landscapes.

3.3 Rastrigin's Function

Rastrigin's function is based on the function of De Jong with the addition of cosine modulation in order to produce frequent local minima. Therefore, the test function is highly multimodal. However, the location of the minima are regularly distributed.[2] The function has the following definition and constrains:

$$f_6(\mathbf{x}) = 10 \cdot n + \sum_{i=1}^n (x_i^2 - 10 \cdot \cos(2\pi x_i)), \quad -5.12 \leq x_i \leq 5.12 \quad (3)$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \dots, n$.

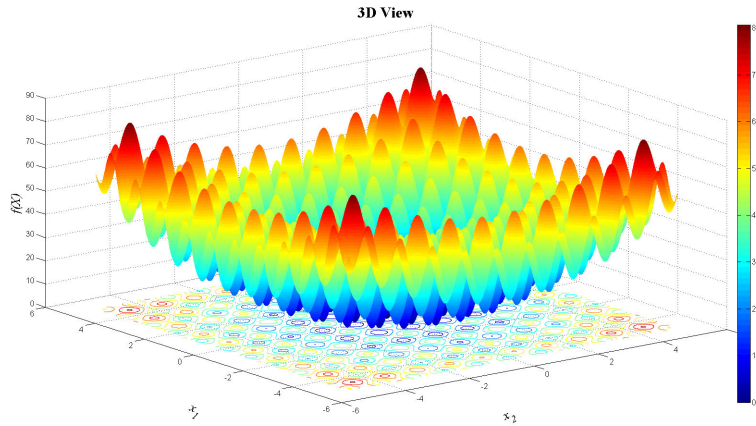


Figure 6: Visualization of Rastrigin's function left with a surface plot for variables in the range $[-5, 5]$.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00016	3.50270	0.48901	0.92920
Dimension = 30	23.77464	50.66660	33.26112	6.06273

Table 9: Statistics for different dimensions using the Genetic Algorithm on Rastrigin’s function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	21.98238	39.50247	27.96103	3.96365
Dimension = 10	33.82661	39.58252	34.74178	1.39452
Dimension = 30	1279.6	3119.3	2008.167	326.7985

Table 10: Statistics for the running time of the Genetic Algorithm(GA) on Rastrigin’s function, in seconds.

Algorithm	Dimension = 5				Dimension = 10			
	Min	Max	Mean	Stdev	Min	Max	Mean	Stdev
HC: FI	3.46661	26.98438	14.31798	6.08346	10.13907	36.06353	23.01735	6.36207
HC: BI	3.4666	15.98912	9.89137	3.08407	7.69236	37.92422	19.66993	6.59668
HC: WI	4.99989	37.24461	14.454	6.46993	17.15856	35.85021	26.06891	4.34619
SA	0.00000	10.89486	3.74039	3.77049	0.00000	11.68616	2.49269	3.09591

Table 11: Heuristic performance of Hill Climbing (HC) and Simulated Annealing (SA) for dimensions 5 and 10 on Rastrigin’s function

Algorithm	Dimension = 30			
	Min	Max	Mean	Stdev
HC: FI	48.82213	92.75589	71.58258	12.23673
HC: BI	36.21646	86.43961	60.52116	11.71494
HC: WI	61.86926	109.4828	83.03921	3.11524
SA	3.70748	22.16531	11.84847	12.03842

Table 12: Heuristic of Hill Climbing (HC) and Simulated Annealing (SA) performance for dimension 30 on Rastrigin’s function

At dimension 5, GA consistently reaches the global optimum with perfect stability, whereas SA shows higher variability. HC variants perform poorly, with high mean values and large deviation. At dimension 10, GA and SA perform similarly, with both near the global optimum. At bigger dimensions, GA’s performance declines due to the complexity of the landscape, whereas SA outperforms GA by reaching lower minima and better mean solutions. These observations are also confirmed by the violin plots presented below.

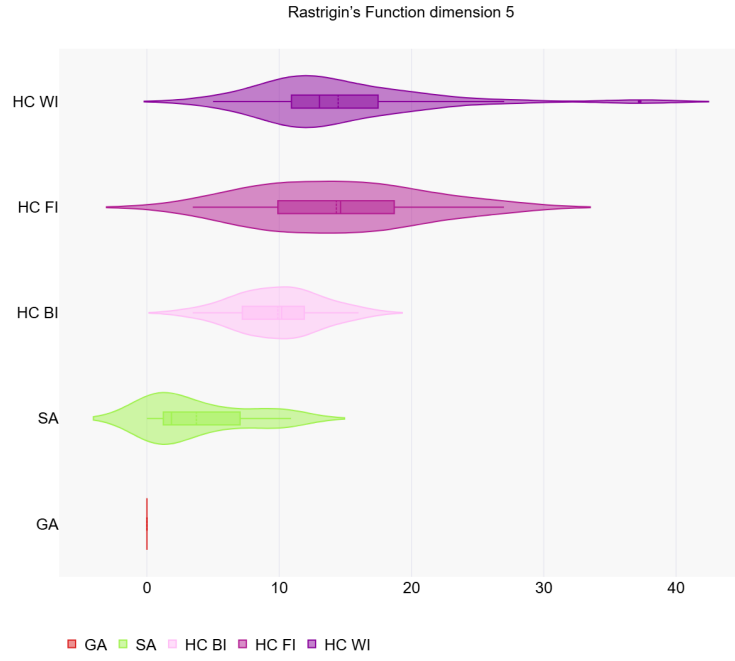


Figure 7: Violin Plot For the algorithms tested on Rastrigin's function, dimension 5.

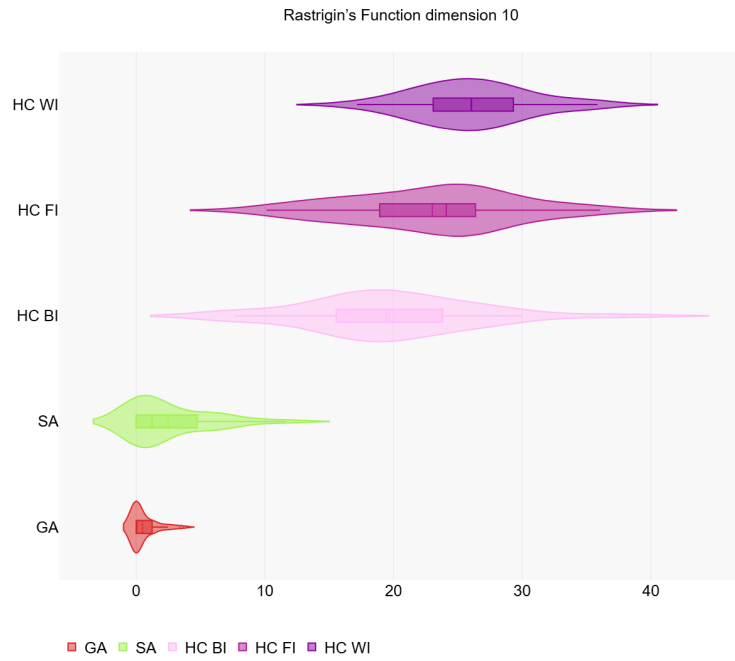


Figure 8: Violin Plot For the algorithms tested on Rastrigin's function, dimension 10.

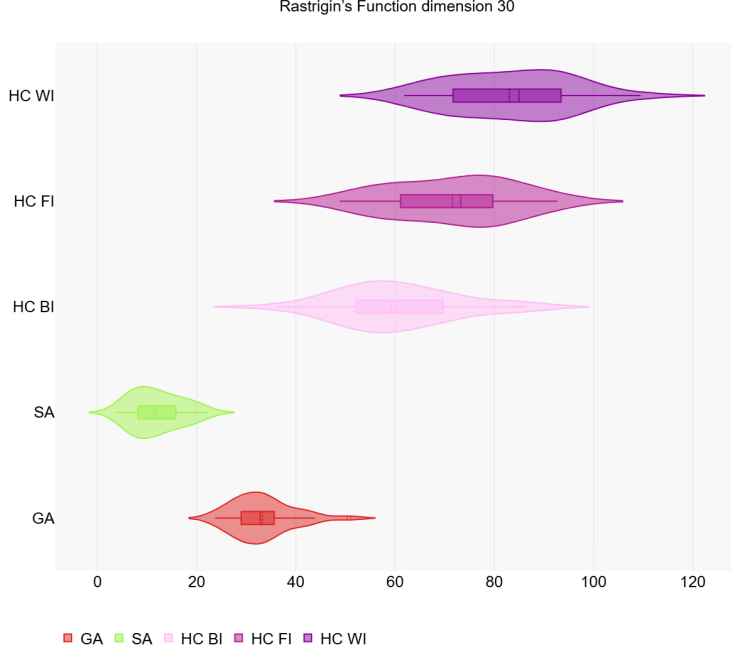


Figure 9: Violin Plot For the algorithms tested on Rastrigin's function, dimension 30.

3.4 Michalewicz's Function

The Michalewicz function is a multimodal benchmark function that possesses $n!$ local optima. [2] The parameter m controls the steepness of the valleys and ridges: higher values of m make the optimization problem increasingly difficult. For large m values, the function exhibits "needle-in-a-haystack" behavior, where most points in the search space provide little information about the location of the global optimum. [3]

$$f_{12}(\mathbf{x}) = - \sum_{i=1}^n \sin(x_i) \left[\sin \left(\frac{i x_i^2}{\pi} \right) \right]^{2m}, \quad i = 1, \dots, n, \quad m = 10, \quad 0 \leq x_i \leq \pi \quad (4)$$

The global minimum value was approximated as $f(x) = -4.687$ for $n = 5$ and $f(x) = -9.66$ for $n = 10$. The corresponding optimal solutions were not provided. [3]

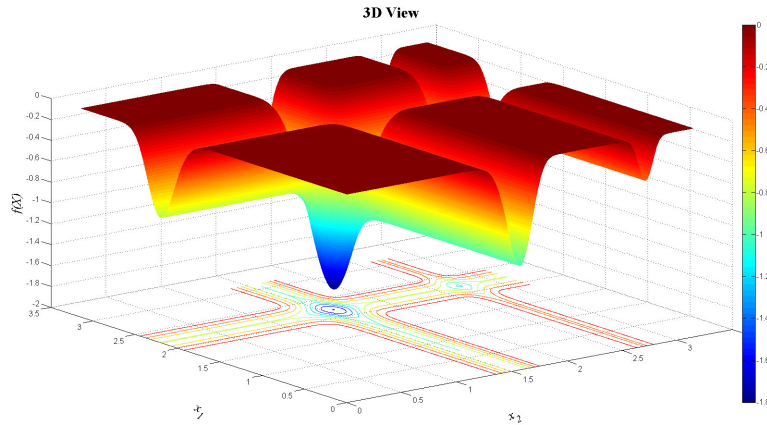


Figure 10: Visualization of the Michalewicz's function. Surface plot for variables x_3 and x_4 over the interval $[0, 3]$, and surface plot for variables x_1 and x_2 over the interval $[0, 3]$.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-4.68766	-4.68593	-4.68754	0.00043
Dimension = 10	-9.65445	-9.32431	-9.50911	0.10053
Dimension = 30	-25.03028	-22.38157	-23.58379	0.54776

Table 13: Statistics for different dimensions using the Genetic Algorithm on Michalewicz's Function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	22.17795	22.83910	22.39582	0.18817
Dimension = 10	36.22302	37.28835	36.55281	0.2429
Dimension = 30	149.27843	151.05406	149.9398	0.37358

Table 14: Statistics for the running time of the Genetic Algorithm on Michalewicz's Function, in seconds.

Algorithm	Dimension = 5				Dimension = 10			
	Min	Max	Mean	Stdev	Min	Max	Mean	Stdev
HC: FI	-4.67025	-2.79532	-3.65719	0.56774	-8.74452	-6.09279	-7.54098	0.81336
HC: BI	-4.53502	-2.98869	-3.92992	0.49091	-9.03804	-6.91331	-8.0361	0.61628
HC: WI	-4.49154	-2.31665	-3.27735	0.58601	-7.67192	-4.47456	-6.25797	0.84421
SA	-4.68766	-4.26747	-4.59811	0.10485	-8.66015	-6.21197	-8.33858	0.45715

Table 15: Heuristic performance of Hill Climbing (HC) and Simulated Annealing (SA) for dimensions 5 and 10 on Michalewicz's Function.

Algorithm	Dimension = 30			
	Min	Max	Mean	Stdev
HC: FI	-25.03433	-19.88829	-22.44486	1.29029
HC: BI	-26.42404	-20.27971	-23.79517	1.38709
HC: WI	-22.17156	-16.54717	-19.01304	1.31556
SA	-27.8186	-26.11362	-27.09268	0.47485

Table 16: Heuristic performance of Hill Climbing (HC) and Simulated Annealing (SA) performance for dimension 30 on Michalewicz's Function

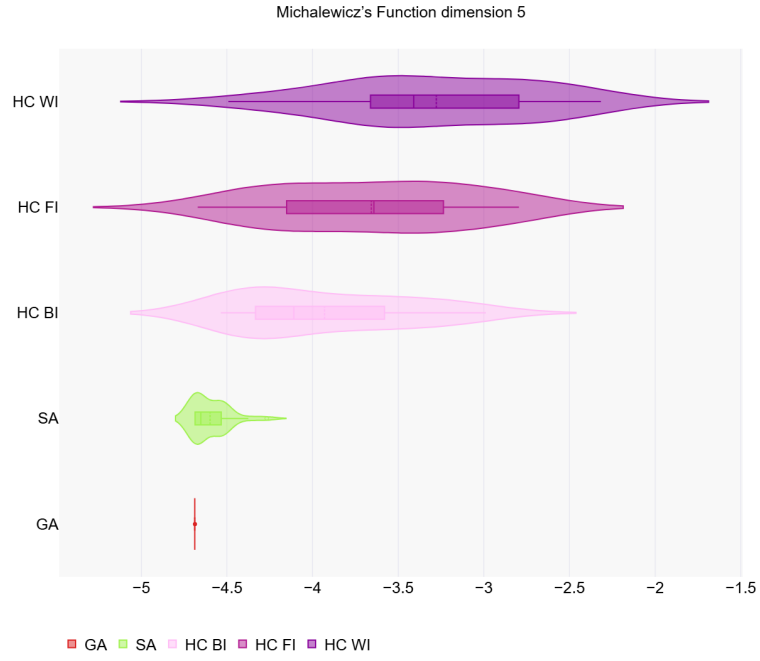


Figure 11: Violin Plot For the algorithms tested on Michalewicz's function, dimension 5.

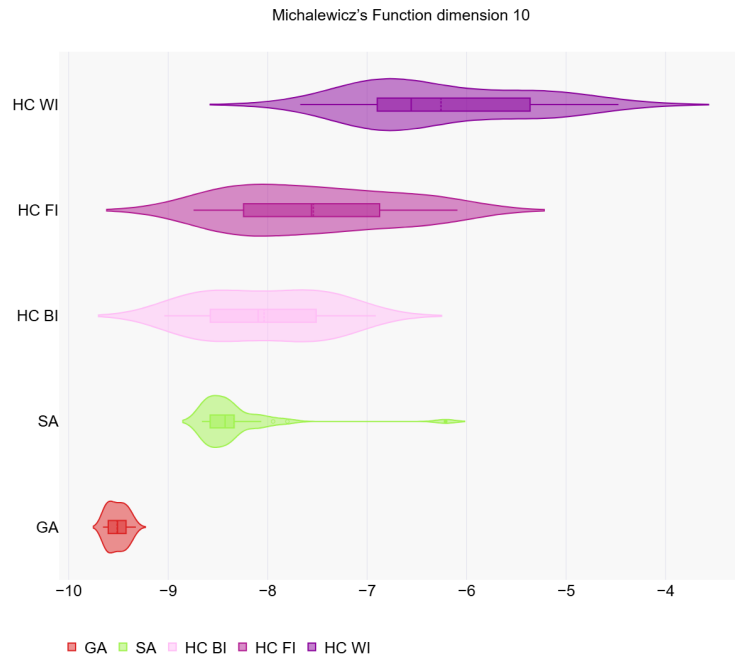


Figure 12: Violin Plot For the algorithms tested on Michalewicz's function, dimension 10.

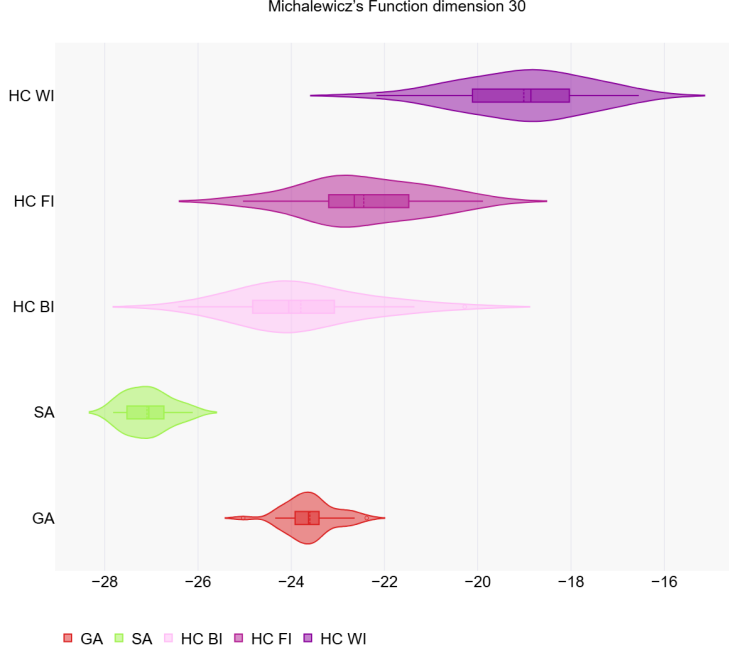


Figure 13: Violin Plot For the algorithms tested on Michalewicz’s function, dimension 30.

GA outperforms all HC variants and SA, reaching closer to the optimum for lower dimensions. GA achieves near-optimal solutions with very low deviation, while SA performs slightly worse, and HC remains unstable with higher mean values. However, at dimension 30, SA surpasses GA in both minimum and mean values, showing superior capability in navigating highly multimodal landscapes. HC, across all dimensions, fails to approach optimal solutions.

Global Performance Insights

Across all benchmark functions, the statistical indicators and violin-plot distributions reveal a consistent pattern in the behavior of the Genetic Algorithm. For low-dimensional settings and functions with smoother landscapes, the GA delivers excellent performance: the distributions are narrow, the variance is minimal, and the best values cluster tightly around the global optimum. This confirms that evolutionary search excels when the search space remains tractable and less fragmented. However, as dimensionality increases, the quality of the GA’s solutions decreases noticeably. The violin plots broaden and the statistical variance grows, indicating a decline in convergence stability. This effect becomes particularly pronounced at *dimension 30*, where the Rastrigin’s and Michalewicz’s functions expose the limitations of the **binary-encoded** GA. In these challenging landscapes, the algorithm struggles to escape complex local structures, ultimately producing results that fall behind those achieved in lower dimensions and, in some cases, even behind the simpler local search strategies(SA) included in the comparison.

Future Work

In future work, the Genetic Algorithm can be improved by integrating strong elements from the other optimization methods studied. From *Hill Climbing*, the algorithm could borrow an efficient local search phase to refine individuals and accelerate convergence. From *Simulated Annealing*, it could incorporate a temperature-based acceptance mechanism to escape local optima and maintain population diversity. Hybridizing GA with HC or SA, together with adaptive mutation rates and better diversity control, may lead to a more robust and consistent optimization method.

4 Conclusions

In this report, we unraveled the performance of the Genetic Algorithm(GA) and compared it with two optimization algorithms, Hill Climbing and Simulated Annealing, on several benchmark functions, including De Jong's, Schwefel's, Rastrigin's, and Michalewicz's. The main goal was to see how the Genetic Algorithm behaves under different levels of complexity and dimensionality and how effectively it can find the global minimum across various types of functions.

The performance analysis of the algorithm demonstrates several consistent patterns. GA consistently finds optimal or near-optimal solutions for low-dimensional functions, such as De Jong, showing performance comparable to or slightly more stable than Hill Climbing and Simulated Annealing. For highly complex functions like Schwefel, Rastrigin, and Michalewicz, GA performs well at low to moderate dimensions, often outperforming HC due to its ability to explore the search space and avoid local minima. However, at higher dimensions, GA's performance may decline slightly compared to SA, which shows superior ability to escape local minima in more complex search spaces.

Overall, the analysis highlights that Genetic Algorithms are highly effective for global optimization in benchmark functions, offering a balance between exploration and exploitation. They outperform local search methods, like Hill Climbing, in most scenarios and remain competitive with Simulated Annealing, particularly at low to medium dimensions. GA is preferable for stable convergence in moderate dimensions, while SA can provide better performance in high-dimensional problems.

Resources and references

- [1] Lect. dr. Eugen Croitoru. *Genetic Algorithms*.
Available at: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>
- [2] *GEATBX: Examples of Objective Functions*.
Genetic and Evolutionary Algorithm Toolbox for use with Matlab. 1994-2000 Hartmut Pohlheim.
Available at: <http://www.geatbx.com/docu/fcnindex-01.html> .
- [3] Marcin Molga, Czesław Smutnicki. *Test functions for optimization needs*. Wrocław University of Technology(3 april 2005)
- [4] GeeksforGeeks. *Introduction to Optimization with Genetic Algorithm*. (2024, September 3). Available at: <https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-optimization-with-genetic-algorithm/>
- [5] Chris Woody Woodruff, "Genetic Algorithms" *Elitism for Evolution Survival of the Fittest*. Fractional architect.(2025, June 26). Available at: <https://www.woodruff.dev/day-12-genetic-algorithms-elitism-for-evolution-survival-of-the-fittest/>
- [6] Algorithm Afternoon.*Fitness Scaling Genetic Algorithm* (2024, April 17). Available at: https://algorithmafternoon.com/genetic/fitness_scaling_genetic_algorithm/
- [7] Baeldung on Computer Science *Genetic Algorithms:crossover probability and mutation probability*. (March 18, 2024). Available at: <https://www.baeldung.com/cs/genetic-algorithms-crossover-probability-and-mutation-probability>
- [8] Alroomi website. *Benchmark functions* .Available at:<https://al-roomi.org/benchmarks/>